

Surface features beside image footprints

Making a footprint map legible with IAU nomenclature, in one coordinate frame

i What this tutorial shows

A footprint map answers “where is there data?”. It does not answer “data over *what?*” — for that you need the named geography. This walks through overlaying IAU-adopted feature names on real image footprints, and using a session-wide target CRS so the two layers are guaranteed to be in the same frame.

Requires the [geo] extra: `pip install "planetariumpy[geo]"`.

The problem this solves is mundane and easy to get wrong. Footprints arrive from one archive in one convention, feature names come from the USGS Gazetteer in another, and nothing stops you from plotting them on the same axes while they quietly disagree.

We work over **Jezero Crater**, Perseverance’s landing site.

1. Pin the frame first

Do this before fetching anything. Everything that follows converts into it, so there is no moment where two layers are in different systems.

```
from planetariumpy import crs

crs.set_target_crs(crs.body_crs("mars"))
crs.get_target_crs().to_authority()
```

```
('IAU_2015', '49900')
```

`body_crs("mars")` resolves to the IAU_2015 ocentric code for Mars — the radii come from the IAU code itself, nothing is hardcoded.

2. Ask where Jezero is

Don't type the coordinates. `nomenclature.find` returns the IAU record for a named feature, so the location is sourced rather than remembered.

```
from planetarypy import nomenclature

jezero = nomenclature.find("mars", "Jezero")
LON = jezero.center_lon.to_value("deg")
LAT = jezero.center_lat.to_value("deg")
LON, LAT, jezero.diameter
```

```
/Users/maye/Dropbox/Documents/01_projects/planetarypy/code/planetarypy/src/planetarypy/nomen
matches = features(body, name=name, **kwargs)
```

```
(np.float64(77.68730000000001), np.float64(18.4082), <Quantity 47.5212 km>)
```

The record carries the feature's own extent too, and the numbers arrive as astropy quantities, so their units travel with them:

```
jezero[["min_lon", "max_lon", "min_lat", "max_lat", "diameter"]]
```

```
min_lon          77.2663 deg
max_lon          78.11120000000001 deg
min_lat          18.0077 deg
max_lat          18.8094 deg
diameter         47.5212 km
Name: 0, dtype: object
```

So the area of interest can be built from the feature rather than from a number someone guessed. We take Jezero's own bounding box and pad it by a few crater diameters of context, converting km to degrees through the body's own radius:

```
import numpy as np
from astropy import units as u

from planetarypy import constants

radius = constants.Mars.mean_radius # a Quantity, in km
deg_per_length = (360 * u.deg) / (2 * np.pi * radius)
```

```
pad = (4 * jezero.diameter * deg_per_length).to(u.deg)
pad
```

3.2131498°

💡 Why units earn their keep here

radius is in **km**. Had it been metres and this code assumed otherwise, the box would have come out 1000× too large — and nothing would have complained. Because both **diameter** and **radius** carry their units, the conversion either works or raises; it cannot quietly produce a wrong number.

Units are on by default. `units.set_units(False)`, or with `units.use_units(False):`, hands back plain floats for code that wants them.

`features` takes plain degrees, so strip the unit at the boundary:

```
pad_deg = pad.to_value(u.deg)

BBOX = (jezero.min_lon.to_value(u.deg) - pad_deg,
        jezero.min_lat.to_value(u.deg) - pad_deg,
        jezero.max_lon.to_value(u.deg) + pad_deg,
        jezero.max_lat.to_value(u.deg) + pad_deg)
WEST_EAST = (BBOX[0], BBOX[2])
SOUTH_NORTH = (BBOX[1], BBOX[3])
BBOX
```

```
(np.float64(74.05315024174811),
 np.float64(14.794550241748105),
 np.float64(81.3243497582519),
 np.float64(22.022549758251895))
```

Nothing above is typed from memory: the location, the size and the body radius are all looked up.

3. The named geography

`nomenclature.features` fetches the gazetteer for a body and caches it. Filters mirror how people actually narrow it: by feature class, by size, by area.

```

import textwrap
import warnings

from planetarypy.crs import CRSConversionWarning

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always", CRSConversionWarning)
    near = nomenclature.features("mars", bbox=BBOX, min_diameter=3)

for w in caught:
    print(textwrap.fill(str(w.message), 78))

```

mars nomenclature reprojected from ESRI:104905 to IAU_2015:49900
 automatically. Pass an explicit CRS, or set
 planetarypy.crs.set_target_crs(...), to choose deliberately.

```
near[["name", "type", "diameter", "center_lon", "center_lat"]].head(8)
```

	name	type	diameter	center_lon	center_lat
0	Jezero	Crater, craters	47.52120	77.6873	18.4082
1	Hargraves	Crater, craters	60.28112	75.7436	20.7371
2	Nili Fossae	Fossa, fossae	727.90630	76.6948	22.0182
3	Angelica	Crater, craters	3.50000	76.9500	18.6500
4	Sedona	Crater, craters	7.40000	77.5400	17.8400
5	Una Vallis	Vallis, valles	5.00000	77.0700	18.2900
6	Neretva Vallis	Vallis, valles	17.00000	77.2000	18.5500
7	Nili Planum	Planum, plana	128.54700	77.1000	18.6000

! You were just told about a conversion

That call emitted a `CRSConversionWarning`. It is not noise — it is the whole point. The gazetteer ships Mars as **ESRI:104905** (GCS_Mars_2000, an ellipsoid). planetarypy standardises on **IAU_2015**. Those differ in more than their label, and a silent reprojection between them is exactly how two datasets end up subtly misaligned with nothing on screen to say so.

Because we set a session target CRS in step 1, the data arrived in the frame we asked for — and said so on the way.

```
near.crs.to_authority()
```

```
('IAU_2015', '49900')
```

Pass an explicit `to_crs=` and the warning goes away: you made the decision yourself, so there is nothing to tell you.

4. Real footprints from an archive

HRSC coverage over the same box, straight from ESA's PSA.

```
from planetarypy import psa

hrsc = psa.query(
    "SELECT TOP 25 granule_uid, c1min, c1max, c2min, c2max "
    "FROM psa.epn_core "
    "WHERE instrument_host_name LIKE '%Mars Express%' "
    "  AND instrument_name LIKE '%HRSC%' "
    f"  AND c1min <= {WEST_EAST[1]} AND c1max >= {WEST_EAST[0]} "
    f"  AND c2min <= {SOUTH_NORTH[1]} AND c2max >= {SOUTH_NORTH[0]} "
    "  AND (c1max - c1min) < 30 AND (c2max - c2min) < 30"
)
len(hrsc)
```

25

Why the < 30 clause

Orbital swaths crossing high latitudes get a degenerate bounding box spanning *all* longitudes. Without that filter you collect scenes that never imaged your area. See the cross-archive tutorial for the full set of PSA rough edges.

5. Put them on one plot

`add_features` is the `coastlines()` move: it draws onto axes you already have, and with `bbox="auto"` (the default) it reads their limits rather than making you restate the area.

```

import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

fig, ax = plt.subplots(figsize=(9, 8))

for row in hrsc:
    ax.add_patch(Rectangle(
        (row["c1min"], row["c2min"]),
        row["c1max"] - row["c1min"], row["c2max"] - row["c2min"],
        fill=False, ec="#1b9e77", lw=0.8, alpha=0.7, zorder=2,
    ))

ax.set_xlim(BBOX[0], BBOX[2])
ax.set_ylim(BBOX[1], BBOX[3])

drawn = nomenclature.add_features(ax, "mars", min_diameter=3)

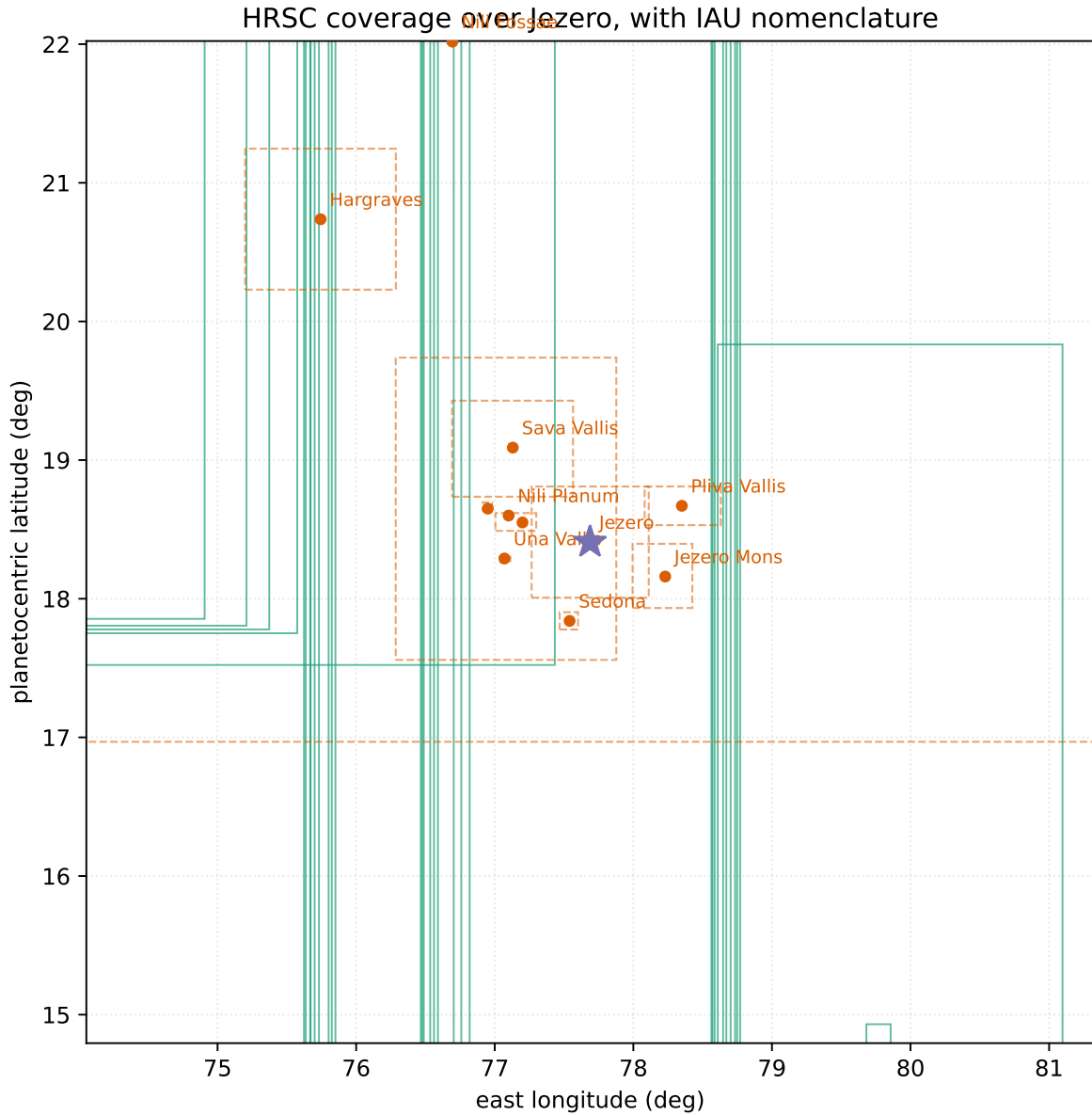
ax.plot(LON, LAT, marker="*", ms=15, color="#7570b3", zorder=7)
ax.set_xlabel("east longitude (deg)")
ax.set_ylabel("planetocentric latitude (deg)")
ax.set_title("HRSC coverage over Jezero, with IAU nomenclature")
ax.grid(alpha=0.3, ls=":")
ax.set_aspect("equal")
plt.show();

```

```

/Users/maye/Dropbox/Documents/01_projects/planetarypy/code/planetarypy/src/planetarypy/nomen
gdf = features(

```



The dashed boxes are feature *extents*, drawn by default from the gazetteer's `min/max_lon` and `min/max_lat` columns. This is the difference that matters: a centre point tells you a name is somewhere nearby, the box tells you whether a given footprint actually covers Jezero or merely passes near it. Pass `extent=False` for centre points only.

Labels are placed largest-feature-first and decluttered — a later label that would collide with one already placed is dropped rather than smeared over it.

```
drawn[["name", "diameter"]].sort_values("diameter", ascending=False).head(6)
```

	name	diameter
2	Nili Fossae	727.90630
7	Nili Planum	128.54700
9	Sava Vallis	62.00000
1	Hargraves	60.28112
0	Jezero	47.52120
8	Pliva Vallis	30.00000

6. Borrowing a different frame for one block

Distances are the usual reason. A geographic CRS measures in degrees, which are not a length; for “how far is this feature from the rover?” you want a local projection in metres. `crs.target_crs` swaps the frame for a block and puts it back afterwards.

```
local = crs.local_crs(LON, LAT, "mars")      # azimuthal equidistant on Jezero

with crs.target_crs(local):
    metric = nomenclature.features("mars", bbox=BBOX, min_diameter=3)
    print("inside the block:", metric.crs.to_authority() or metric.crs.name)

print("after the block: ", crs.get_target_crs().to_authority())
```

```
inside the block: AzimuthalEquidistant(18.4082, 77.6873) on mars
after the block:  ('IAU_2015', '49900')
```

```
/var/folders/y_/4r7grswx4x3dy3s2c0z3h8180000gn/T/ipykernel_2172/193503855.py:4: CRSConversionWarning
    metric = nomenclature.features("mars", bbox=BBOX, min_diameter=3)
```

The session setting is restored on exit — and would be even if the block had raised. Now distances are real:

```
from shapely.geometry import Point

origin = Point(0, 0)    # the projection is centred on Jezero, so this is the site
metric["km_from_site"] = metric.geometry.distance(origin) / 1000
metric.nsmallest(6, "km_from_site")[["name", "km_from_site"]]
```


	name	km_from_site
0	Jezero	0.000000
6	Neretva Vallis	28.655637
10	Jezero Mons	33.902695
4	Sedona	34.687030
7	Nili Planum	34.915039
5	Una Vallis	35.429514

7. Tidy up

```
crs.clear_target_crs()
crs.get_target_crs() is None
```

True

Leaving a session CRS set is harmless in a notebook, but a library function that sets one and never clears it would surprise its caller. Prefer the context manager where the scope is known.

Where to go next

- `planetariumpy.nomenclature.bodies()` lists all 47 bodies the gazetteer covers — the Moon has 9086 named features, Mercury 606, Europa 129.
- `planetariumpy.units` toggles astropy units project-wide, with the same set/context-manager pair as the target CRS.
- The HOWTO on session CRS covers precedence and silencing the conversion notice.
- The cross-archive tutorial combines PDS and PSA discovery over one area.